

Docker → Used to Isolate Service

classmate

Date _____
Page 20

- Before K8s, we have to learn to rewire docker & containerization, so here is everything about docker:-

Docker

- It is used to containerize our code, dependencies, making it OS independent. It is light weight & fast.
 - It contains its own OS & filesystem.
- Docker container contains everything.
 - It's an running instance.
 - Very light weight than extra server & physical machine.
- Docker image: With help of images, containers are created. Blueprint to create container.
- What's sent is docker image.
- Dockerfile: Dockerfile is used to create this images.
 - Every instructions is a layer stacked on each other.
 - Only the top layer is executable.
 - Every other layer is read only.
- This image stack makes it light weight & portable.
- Docker-compose: Used to run multi-containers.
- In real world, we don't have a single container to run, it's multiple.

→ For microservice project, it is better to containerize & run it!

classmate

Date

Page

71

→ Each service has its own container, isolation & many optimize so docker-compose file helps there.

It is written in YAML format.

- Containerization: Combination of (multiple) Dockerfile + (single) Docker-compose file

which creates image & run multiple images is called containerization.

- Dockerhub Just like GitHub, repository for your images.

- Also consists of famous useful images like node, mysql, maven, tomcat and many more.

- Docker Engine: Core part of Docker that runs & manages containers.

- Docker volumes: Containers don't store data, to store data permanently, volumes are used.

→ You can host websites on docker containers too

→ can run multiple containers using compose file & build a microservice project.

→ Containers have their own IP address

All useful commands:

- `docker pull image-name` → Pulls inbuilt image
- `docker images` → shows existing images
- `docker run image-name` → Make container
- `docker ps -a` → show all container
- `docker stop container-name` → stops
- `docker start container-name` → restarts
- After stopping container,
`docker rm container-name` - deletes container
- `docker rmi image-name` → deletes images.
- `docker login` - To login to dockerhub
- `docker push image dockerhub-username: image-name`
↳ Push to dockerhub, image.
- `-d` → detach mode
→ As container is an running process,
might take up your terminal &
stops your use to terminal so
to run the container in background
use detach mode.
- `-it` - Interactive mode
- to run bash scripts & react with
containers.

- docker inspect container-name

→ To inspect & get data like IP of container

classmate

Date

Page

73

- exec - docker container are not much changeable

- By use of -exec command you can go inside container, to its directory to change something.

- docker compose up → Runs your compose file & create & run & deploy multicontainers

- docker compose down → stops & delete all the containers created by compose file.

- docker logs → To see the logs, you don't know what's happening in docker so logs helps alot.

- docker build -t ^{image-}name

- Build image from Dockerfile in current directory

- docker network ls - Lists network

- docker volume ls - Lists volumes

- docker volume create volume-name
↳ create volume

- docker volume rm <volume-name>

- docker tag <image-id> <repo:tag>
↳ To tag image

- docker run -p 8080:80 <image>

- prune → used to clean up host or container port
↳ port binding

Containers are OS virtualization ^{Don't need OS}
VMS are hardware-virtualization ^{classmate}
- Need OS

Docker Port Binding:-

- When you run a container, it has its own internal ports

[Eg:- a web app might run on port-80 inside a container]

- Port binding allows you to connect that container port to your [host] port so you / user can access the app from browser.

`docker run -p 8080:80 <image-name>`

↑ ↑

host container-IP

- nginx listens to port 80

So, you can access nginx service at your public-IP:8080

Dockerfile [To create image]

FROM - Takes built-in image from Dockerhub

LABEL - To tag metadata

RUN - To perform ^{shell} instructions

COPY - copy files from local to image

ADD - Like COPY but unpack zip/tar file

WORKDIR - Set working directory of container

ENV - set environment variables

EXPOSE - Documents the port container listens to

CMD - Single / Default command to run at when container starts

More
probably
than CMD

- in list point.

ENTRYPOINT - main command to run, even if arguments are passed.

VOLUME - creates a mount point with a volume.

Docker compose file [To run multiple containers]

↳ YAML based

version: '3.8'

services:

-- web:

-- image: nginx:latest

ports:

-- "8080:80"

volumes:

-- ./html:/usr/share/nginx/html

restart: always

-- db:

-- image: mysql:8.0

environment:

-- MYSQL_ROOT_PASSWORD: Pass

-- MYSQL_DATABASE: mydb

ports:

-- "3306:3306"

volumes:

-- db-data:/var/lib/mysql

volumes:

-- db-data:

↳ Already created volume

↳ It automatically create

volume if it doesn't exist.